Project1 实验报告

多项式与表达式求值计算器

学号：25345110 姓名：王志轩

一、 程序功能简要说明

1.1 整体功能与文件组织

本程序使用 C++ 实现多项式计算与表达式求值，提供终端和 Qt 图形界面两种入口。图形界面左侧显示过程与提示，右侧上方显示输入和结果、下方放置数字键盘，顶部按钮切换模式。核心链表和栈均手工实现，两种界面复用同一组计算函数。

文件或目录	主要职责
`main.cpp`	终端菜单、提示和输入恢复。
`polynomial.h/.cpp`	多项式结构、输入输出、加减乘、求导、求值与释放。
`expression.h/.cpp`	模板栈、优先关系、数字识别和表达式求值。
`gui/`	Qt 窗口、按钮事件与自然多项式输入适配。
`CMakeLists.txt`	共用核心库及图形、终端可执行程序的构建配置。

1.2 多项式计算器

支持输入输出、加减乘、求导和指定 x 处求值。终端先输入项数，再按指数严格降序输入整数系数与非负指数；零多项式输入项数 0。图形界面直接输入 `2x^3+5x`、`x+1` 或 `0`，无需项数，适配函数负责整理顺序、合并同类项。结果省略首项正号、含 x 项的单位系数与一次幂。求值时选择“求值”并点击 x 输入框，才启用小数点键；表达式与多项式系数仍使用整数。

1.3 表达式求值与扩展功能

支持整数的 `+ - * /`、括号及普通空格，除法向零截断，右结合的乘方 `^`，一元负号仅允许在开头或左括号后，内部记为 `~`，不接受省略乘号的 `2(3+4)`。双栈过程显示在左侧，可滚动查看。两种模式分别保留输入和结果；回车或点击 `=` 执行计算。

1.4 异常处理与当前边界

已检查除零、空括号、括号不匹配、缺操作数或运算符、非法字符、负指数及栈满等情况。每个栈容量为 100。图形程序在 Apple Silicon Mac 上验证，未在其他平台实测。

二、 程序运行截图

2.1 代表性运行结果

下列 23 个代表性算例来自核心程序的实际运行验证，可用于对照两种界面的结果。表中“失败”表示求值函数返回失败，由菜单输出错误提示。图形界面另已验证模式切换、屏幕键盘、回车计算、五种多项式操作与错误输入后的继续计算。

表 1 多项式基本功能的实际结果

功能	输入多项式或操作	实际结果
加法	$(2x^3+5x)+(3x^3+4)$	$5x^3+5x+4$
相消	$(3x^4+2x^2)+(-3x^4-2x^2)$	0
减法	$x^2-(x^2+x)$	$-x$
乘法	$(x+1)(x-1)$	x^2-1
求导	$(3x^3+2x^2+7)'$	$9x^2+4x$
求值	$f(x)=2x^2,\ x=2$	$f(2)=8$

表 2 基础表达式及扩展运算的实际结果

表达式	实际结果
`10-7`	3
`4*6`	24
`10-2*3`	4
`2*3+4*5`	26
`(2+3)*4`	20
`((2+3)*4+1)/3`	7
`20/4/5`	1
`2^3^2`	512
`-3+2`	-1
`3*(-2)`	-6
`-3^2`	-9
`(-3)^2`	9

表 3 异常表达式的实际处理

表达式	原因	实际结果
`1/0`	除数为零	失败
`(2+3`	括号不匹配	失败
`()9+1`	空括号	失败
`3*-2`	一元负号位置不符合约定	失败
`2^(-3)`	负指数不属于当前整数计算范围	失败

2.2 图形界面整体布局



图 1　Qt 图形界面整体布局与模式切换入口

切换到多项式模式后，右侧显示 A、B、五种操作按钮及 x 输入框。当前不需要的字段和按键会禁用，例如求导不需要 B，整数表达式不启用小数点键。数字键盘写入最近点击的输入框，避免误把 x 的取值输入到多项式中。

2.3 多项式运行截图



图 2　图形界面的多项式加减乘演示



图 3　图形界面的求导

2.4　拓展运算集与异常输入截图



图 4　图形界面的乘方与一元负号演示



图 5　图形界面中的错误提示

三、 部分关键代码及其说明

3.1 带头节点单链表及多项式加减

多项式节点只存储一项，头节点不参与计算。实际项从 `head->next` 开始。`length` 在初始化时暂存待输入项数，输入函数保存该数后将其置零，随后每成功建立一个节点增加一次，因此输入失败时也能反映已建立的节点数。

```
struct Term {
    int exp;
    int coef;
    Term* next;
};
struct Polynomial {
    int length;
    Term* head;
};
```

加减法同时遍历两条按指数降序排列的链表。指数较大的一项先复制到结果，只推进对应指针；指数相同时合并系数，并同时推进两个指针。以下为加法中同指数项的处理：

```
int coef = term1->coef + term2->coef;
if (coef != 0) {
    tail->next = new Term{term1->exp, coef, nullptr};
    tail = tail->next;
    result.length++;
}
term1 = term1->next;
term2 = term2->next;
```

系数为零时不创建节点，也不移动结果链表的尾指针。其中一条链遍历结束后，继续复制另一条的剩余项。减法的过程相同，但来自第二个多项式的系数需要取反。若两条链分别有 m、n 项，加减法时间复杂度为 $O(m+n)$。

3.2 乘法、求导、求值与资源释放

乘法采用“单项乘整个多项式，再累加”的方法：用 a 的每一项乘 b 的所有项形成临时链表 `partial`，再调用加法合并结果。临时链表保持指数降序，因此可以复用加法函数。新结果由独立节点组成，必须先保存新结果，再释放旧结果和临时链表：

```
Polynomial sum = addPolynomial(result, partial);
deletePolynomial(result);
deletePolynomial(partial);
result = sum;
```

这里的结构体赋值传递的是链表地址，不会自动复制节点；`sum` 是独立的新链表，上述赋值用于把它交给 `result` 继续管理。删除函数沿指针遍历，连同头节点一起释放，最后将头指针置空。

求导逐项执行 $(cx^e)' = cex^{e-1}$，常数项直接跳过；求值逐项累加 cx^e。当前乘法还包含反复合并和释放链表的开销，最坏时间复杂度可达 $O(m^2n)$，其中 m 是外层遍历的 a 的项数，n 是 b 的项数；尚未针对大量项进行优化。

3.3 模板顺序栈与运算符优先关系

`Stack<int>` 与 `Stack<char>` 分别用于运算数和运算符，共用模板栈操作。`top=-1` 表示空栈；入栈前检查容量，出栈和查看栈顶前检查是否为空。`pop` 与 `peek` 使用引用参数带回元素，返回值表示是否成功。

通过观察规律得出来乘方 `^` 与一元负号 `~` 的运算优先级关系从而拓展了运算符集，表中行是栈顶运算符，列是当前输入运算符。`x` 表示不合法组合。`<` 表示输入符号入栈并前进，`>` 表示先计算栈顶运算，`=` 表示括号匹配或结束。

表 4 扩展后的算符优先关系表

栈顶 / 输入	+	−	*	/	(	)	#	^	~
+	>	>	<	<	<	>	>	<	x
−	>	>	<	<	<	>	>	<	x
*	>	>	>	>	<	>	>	<	x
/	>	>	>	>	<	>	>	<	x
(	<	<	<	<	<	=	x	<	<
)	>	>	>	>	x	>	>	>	x
#	<	<	<	<	<	x	=	<	<
^	>	>	>	>	<	>	>	<	x
~	>	>	>	>	<	>	>	<	x

当关系为 `>` 时，**不推进输入下标**，因为当前输入还要继续与新的栈顶比较。例如读取 `10-2*3` 末尾的 `#` 时，应先算 2×3，再算 $10 - 6$，之后才完成终止符匹配。二元运算先弹出右操作数，再弹出左操作数，计算顺序不能颠倒。

3.4 数字识别与一元负号

`getNumber` 按“旧值乘 10，再加当前数字”读取整数并推进下标。

`expectOperand` 记录是否等待操作数；合法的一元负号转为 `~`。下列摘录分别处理一元取负与二元运算。

```
int value;
if (op == '~') {
    int operand;
    if (!pop(numberStack, operand)
        || operand == std::numeric_limits<int>::min()) {
        return false;
    }
    value = -operand;
} else {
    int right, left;
    if (!pop(numberStack, right) || !pop(numberStack, left)
        || !calculate(left, op, right, value)) {
        return false;
    }
}
if (!push(numberStack, value)) {
    return false;
}
```

结束时，运算数栈必须恰好剩一个元素，才作为结果返回。

3.5 Qt 界面、按钮事件与算法复用

图形入口使用 QApplication 管理事件循环。窗口类继承 QWidget，通过水平、垂直和网格布局安排左右区域、输入结果及按键。程序显示窗口后进入事件循环，由点击或键盘事件触发处理函数，不再使用终端输入循环。入口的核心代码如下：

```
QApplication application(argc, argv);
CalculatorWindow window;
window.show();
return application.exec();
```

模式按钮的点击信号连接到回调函数，回调改变模式并刷新控件：

```
connect(modeButton, &QPushButton::clicked, this, [this]() {
    polynomialMode = !polynomialMode;
    updateMode();
});
```

多项式操作按钮使用互斥的 QButtonGroup，并跟随 toggled 信号更新实际操作，保证选中状态与执行的加、减、乘、求导或求值保持一致。eventFilter 在输入框获得焦点时记录目标，数字键盘通过 activeInput 写入字符。数字按钮不抢走输入焦点；C 重置当前模式，退格键删除光标前内容。

原函数新增输入、输出流参数，保留面向终端的旧接口。界面将表达式运算过程写入字符串流，再显示到左侧文本框：

```
std::ostringstream trace;
int result;
bool success = evaluateExpression(expression.toStdString(),
                                  result, trace);
QString details = QString::fromStdString(trace.str());
```

成功时更新结果框与过程框，失败时显示错误提示。多项式输入适配函数逐项识别系数和指数，用字符串流交给原 inputPolynomial 构造单项，再复用 addPolynomial 整理顺序并合并同类项，因此图形输入不要求用户提前给项数。输出同样复用 printPolynomial，核心计算没有改成另一套界面算法。Qt 父控件负责管理子控件生命周期，计算产生的链表仍由原删除函数释放。

四、 程序运行方式简要说明

4.1 文件结构

两个入口分别编译，每个版本均支持多项式计算和表达式求值，并共用核心算法。

文件或目录	作用
`main.cpp`	终端入口：显示文字菜单，通过终端输入和输出。
`gui/main.cpp`	图形入口：创建 Qt 应用，显示计算器窗口。
`polynomial.h/.cpp`、`expression.h/.cpp`	两个入口共用的多项式与表达式算法。
`gui/` 中其余文件	窗口、按钮事件及多项式文本输入适配。
`CMakeLists.txt`、`run_gui.command`	图形版本的构建配置与 Mac 启动脚本。
`README.md`、`report/`	运行说明与实验报告。

4.2 环境与依赖版本

- 操作系统：macOS 14.5，Apple Silicon。
- 编译器：Apple clang 16.0.0；终端版 C++11，图形版 C++17。
- 界面库：Qt Widgets 6.8.3。
- 构建工具：CMake 3.21 或以上，本机版本为 4.4.4。
- 可选安装工具：Python 3.9 或以上、aqtinstall 3.3.0，仅用于安装 Qt 和构建工具。

4.3 启动入口

以下命令在项目根目录执行。

终端版：`main.cpp`

```
clang++ -std=c++11 main.cpp polynomial.cpp expression.cpp -o calculator_cli
./calculator_cli
```

图形版：`gui/main.cpp`

```
./run_gui.command
```